

- AI工具使用披露说明：
 - 1. 使用工具列表
 - 2. 各类工具的具体作用
 - 2.1 文献调研与综述撰写
 - 2.2 代码生成、重构与性能优化（最主要使用场景）
 - 2.4 Prompt 工程与 LLM 调用代码
 - 2.5 报告撰写与结构优化
 - 3. 局限性与真实失败案例
 - 3.1 第一阶段 RGB-D 坐标转换问题
 - 3.2 启动与运行性能问题
 - 5. 提示词使用情况：
 - （1）提示词记录1：
 - （1）提示词记录2：

AI工具使用披露说明：

1. 使用工具列表

- 大语言模型（LLM）：
 - GPT-4o
 - DeepSeek-V3 / DeepSeek-R1
 - Grok（xAI）
 - Claude 3.5 Sonnet
- Coding Agent：
 - Cursor
 - GitHub Copilot

2. 各类工具的具体作用

2.1 文献调研与综述撰写

第一阶段使用 GPT-4o + DeepSeek 调研 YOLO + RGB-D 视觉避障 / 语义导航相关工作，最终确定本项目技术路线。

2.2 代码生成、重构与性能优化（最主要使用场景）

- Cursor Agent 完成 YOLO + RGB-D 相机坐标转换、目标点投影到 map 坐标系的实现与修复。
- 重构整个项目仓库结构，增加 `scripts/` 一键启动脚本、模块化节点拆分、日志系统。
- 生成并优化 `command_executor_node.py`、`task_manager_node.py`、`patrol_smallhouse.py` 等核心节点。
- Nav2 参数调优、RViz 性能优化（解决 waffle 模型渲染导致的帧率下降问题）。

2.4 Prompt 工程与 LLM 调用代码

Cursor 协助设计 `llm_router_node` 的系统提示词（System Prompt），定义 JSON 输出格式（command、args、rationale_zh、confidence），并实现云端 DeepSeek API + 本地 Whisper 离线回退机制。

2.5 报告撰写与结构优化

使用 Grok 帮助梳理和优化报告整体结构与逻辑，撰写并多次迭代，引言，应用场景，量化评估指标，系统与部署分析，风险、局限与未来工作，等章节；

3. 局限性与真实失败案例

尽管 AI 工具极大提升了开发效率，但仍存在以下局限性和实际问题：

3.1 第一阶段 RGB-D 坐标转换问题

初期 Cursor 生成的坐标转换代码存在 TF 变换时序问题，导致目标点投影不准确。最后因时间限制，暂时使用普通相机+角度估算替代。**第二阶段通过 Cursor 参考高质量同类仓库 + 多次迭代验证**，才在第三次尝试后实现正确的 map 坐标投影。

3.2 启动与运行性能问题

项目早期启动缓慢（超过 40 秒）。经 Cursor 增加各节点启动计时 + 详细日志后，发现主要瓶颈是 RViz 中 waffle 机器人模型在开启深度相机时的渲染开销（远高于 burger 模型）。此问题 AI 无法直接定位，最终通过人工 + 日志分析解决，启动时间提升约一倍。

5. 提示词使用情况：

（1）提示词记录1：

你现在在一个 ROS 2 Humble + TurtleBot3 + Gazebo + Nav2 的机器人项目中工作。

当前项目已经存在如下结构：

```
robot_bringup/
├── config/
├── launch/
├── maps/
│   ├── map.yaml
│   └── map.pgm
├── models/
├── scripts/
├── urdf/
└── world/
```

```
robot_navigation/
├── resource/
├── robot_navigation/
├── package.xml
└── setup.py
```

目前已经确认：

1. robot_bringup/maps/map.yaml 和 map.pgm 是已经建好的静态地图。
2. 当前目标不是 SLAM 建图，而是：
“基于已有地图实现一个初版 Nav2 导航系统”。
3. 项目未来还会继续扩展：
 - YOLO
 - 动态障碍剔除
 - 自然语言交互
 - 任务系统
 - 房间搜索
 - 行为树
 - Agent 系统

因此需要保持工程结构清晰，不要把所有东西都堆到 bringup 中。

=====

一、总体架构要求

=====

请按职责划分：

1. robot_bringup

负责：

- Gazebo
- 机器人模型
- 世界
- 地图资源
- 系统级 bringup

2. robot_navigation

负责：

- Nav2
- AMCL
- 导航参数
- navigation launch

- 导航运行脚本

不要把 Nav2 的主要配置直接堆到 robot_bringup。

=====

二、需要新增的目录结构

=====

请在 robot_navigation 下补齐：

```
robot_navigation/  
├─ config/  
│   └─ nav2_params.yaml  
├─ launch/  
│   └─ navigation.launch.py  
├─ scripts/  
│   └─ run_nav2.sh  
├─ resource/  
├─ robot_navigation/  
├─ package.xml  
├─ setup.py  
└─ README.md
```

如果 setup.py、package.xml 缺少 launch/config/scripts 安装规则，请补齐。

=====

三、地图使用方式

=====

不要重新建图。

必须直接复用：

```
robot_bringup/maps/map.yaml  
robot_bringup/maps/map.pgm
```

navigation.launch.py 中默认引用：

```
robot_bringup/maps/map.yaml
```

不要复制地图文件。

=====

四、Nav2 功能要求

=====

实现一个最小可运行的 Nav2 导航闭环：

Gazebo 启动

- 加载静态地图
- AMCL 定位
- RViz 设置初始位姿
- Nav2 Goal 导航
- 机器人移动

要求：

1. 不启动 SLAM
2. 使用 map_server
3. 使用 AMCL
4. 使用 planner_server
5. 使用 controller_server
6. 使用 bt_navigator
7. 使用 behavior_server
8. 使用 waypoint_follower
9. 使用 lifecycle_manager

=====
五、navigation.launch.py 要求
=====

请实现：

robot_navigation/launch/navigation.launch.py

要求：

1. 使用 ROS 2 Python Launch 写法
2. 支持参数：
 - use_sim_time
 - map
 - params_file

3. 默认值：

use_sim_time := true

map :=
robot_bringup/maps/map.yaml

params_file :=
robot_navigation/config/nav2_params.yaml

4. 优先复用：
nav2_bringup/bringup_launch.py

5. 必须确保：

slam := False

6. launch 文件尽量简洁，
优先 include 官方 Nav2 bringup。

=====
六、nav2_params.yaml 要求
=====

新增：

robot_navigation/config/nav2_params.yaml

要求：

1. 参数适合 TurtleBot3 仿真

2. 保持“初版导航”级别，不要过度复杂

3. 配置：

- AMCL
- planner
- controller
- local_costmap
- global_costmap
- lifecycle_manager

4. TF frame 必须统一检查：

- map
- odom
- base_link 或 base_footprint
- scan

如果当前项目使用 `base_footprint`，
不要强行改成 `base_link`。

=====

七、`run_nav2.sh` 要求

=====

新增：

`robot_navigation/scripts/run_nav2.sh`

脚本要求：

1. `source ROS 2 Humble`
2. `source` 当前工作区 `install/setup.bash`
3. 设置 `TURTLEBOT3_MODEL`
4. 启动：
 - Gazebo
 - `robot bringup`
 - Nav2
 - RViz（如果已有）

5. 尽量不要写死绝对路径

6. 输出清晰日志

=====

八、`robot_bringup` 的职责

=====

`robot_bringup` 不负责实现 Nav2 细节。

如果需要：

只允许新增一个“很薄”的总 `launch`：

`robot_bringup/launch/system_navigation.launch.py`

功能仅限：

`include robot_navigation/navigation.launch.py`

不要把 Nav2 参数直接写在 bringup 中。

=====

九、README 要求

请补充运行说明：

1. 编译：

```
colcon build
source install/setup.bash
```

2. 启动：

```
bash robot_navigation/scripts/run_nav2.sh
```

3. RViz 操作：

- 确认 /map 显示
- 2D Pose Estimate
- Nav2 Goal

4. 调试命令：

```
ros2 topic list
ros2 topic echo /map --once
ros2 topic echo /scan --once
ros2 topic echo /amcl_pose
ros2 lifecycle nodes
ros2 lifecycle get /map_server
ros2 lifecycle get /amcl
```

5. TF 检查：

```
map
→ odom
→ base_footprint/base_link
```

=====

十、重要约束

1. 不要删除已有 SLAM/YOLO/strip 相关代码
2. 不要大规模重构项目
3. 只实现“已有地图 + AMCL + Nav2”
4. 保持工程化结构
5. 小步修改
6. 所有新增文件都符合 ROS 2 工程规范

=====

十一、最终输出

修改完成后，请输出：

1. 新增了哪些文件

2. 修改了哪些文件
3. 每个文件的作用
4. 启动命令
5. 预期运行现象
6. 如果失败，优先检查哪些 topic / TF / lifecycle 节点

最终目标：

robot_navigation 成为独立导航模块；
robot_bringup 负责系统 bringup；
地图复用 robot_bringup/maps/map.yaml；
实现一个真正可运行的初版 Nav2 导航系统。

(1) 提示词记录2：

你现在是一个资深 ROS2 机器人系统架构师和工程重构专家。请你帮助我把当前机器人项目重构成一个真实、工程化、可持续开发的完整机器人系统，而不是只整理现有脚本。

一、项目背景

当前项目基于 ROS2 Humble、Gazebo、TurtleBot3、SLAM Toolbox、Nav2、YOLO 等组件，已有内容主要包括：

1. 仿真环境运行；
2. SLAM 建图；
3. baseline 普通建图流程；
4. experiment AI 语义辅助建图流程；
5. YOLO/person 检测；
6. 动态行人区域标记与累积；
7. before_strip 地图保存；
8. strip_saved_map_person_free 后处理；
9. after_strip 地图输出；
10. 部分 shell 脚本用于启动项目。

但是项目未来不只是建图实验，而是要发展成一个完整的室内服务机器人系统，后续还会接入：

1. 语音识别 ASR；
2. 自然语言理解 NLU；
3. 任务规划 Task Planning；
4. Nav2 导航；
5. 屋内目标搜索；
6. YOLO 物体识别；
7. 人物/动态障碍物识别；
8. 语音交互；
9. 一键运行 shell 脚本；
10. 多场景实验与评测。

因此，请不要只按“SLAM 项目”来整理，而要按“完整机器人系统工程”来设计架构。

二、总体目标

请将当前项目重构为一个清晰分层、模块解耦、可持续扩展的机器人系统架构，使项目能够支持以下能力：

1. 仿真启动;
2. 建图;
3. 地图后处理;
4. 导航;
5. 语音识别;
6. 自然语言理解;
7. 任务分发;
8. 屋内搜索;
9. YOLO 视觉识别;
10. 实验运行;
11. 日志记录;
12. 参数配置;
13. shell 一键启动;
14. 后续模块持续接入。

三、请优先采用如下工程分层思想

项目整体可按以下层次组织:

1. apps: 系统入口层
 - 放完整运行入口;
 - 包括 `baseline`、`experiment`、`demo`、`simulation`、`search_task` 等入口;
 - 负责把多个模块组合成可运行流程。
2. bringup: 机器人启动层
 - 放 ROS2 launch 文件;
 - 负责启动 Gazebo、`robot_state_publisher`、SLAM、Nav2、YOLO、ASR、NLU、任务管理等节点;
 - 不放具体算法逻辑。
3. core: 核心数据结构与公共接口层
 - 定义统一的任务、事件、状态、动作、消息封装;
 - 后续 ASR、NLU、导航、搜索、视觉模块都通过统一接口协作;
 - 不依赖具体算法。
4. perception: 感知层
 - lidar: 激光雷达输入处理;
 - vision: 摄像头、YOLO、目标检测;
 - speech: ASR 语音识别;
 - fusion: 多模态信息融合;
 - 输出感知结果, 例如 `person_detected`、`object_detected`、`speech_text`。
5. mapping: 建图层
 - SLAM Toolbox 相关封装;
 - semantic map marker;
 - 地图保存;
 - `before_strip` 输出;
 - 地图元数据管理。
6. processing: 后处理与算法增强层
 - strip 地图后处理;
 - person region 累积;
 - dynamic object filter;
 - map clean/filter/refinement;
 - 不直接负责系统启动。
7. navigation: 导航层

- Nav2 启动与封装;
- goal sender;
- path planner wrapper;
- navigation monitor;
- 后续支持“去客厅”“去厨房”“搜索某物”等任务。

8. interaction: 人机交互层

- ASR 结果处理;
- NLU 解析;
- TTS 输出;
- command parser;
- dialog manager;
- 将自然语言转为机器人任务。

9. task: 任务管理层

- task manager;
- task planner;
- search task;
- patrol task;
- navigation task;
- map building task;
- 将高层任务拆解为感知、导航、识别等子任务。

10. experiment: 实验与评测层

- baseline vs experiment;
- map quality evaluation;
- search success rate;
- navigation success rate;
- logs/results/metrics;
- 保证课程实验可复现。

11. scripts: 脚本层

- shell 脚本;
- setup 脚本;
- run 脚本;
- clean 脚本;
- evaluation 脚本;
- 不要把核心业务逻辑全堆在 scripts 里。

12. config: 统一配置层

- slam.yaml;
- nav2.yaml;
- yolo.yaml;
- asr.yaml;
- nlu.yaml;
- task.yaml;
- experiment.yaml;
- paths.yaml;
- 所有阈值、模型路径、topic 名称、输出路径尽量集中管理。

13. data: 数据与输出层

- maps;
- logs;
- results;
- datasets;
- recordings;
- 不放代码。

14. docs: 文档层

- architecture.md;
- runbook.md;
- experiment.md;
- module_design.md;
- README.md。

四、推荐目标目录结构

请你结合当前项目实际情况，尽量重构成如下结构：

```
robot_project/
├── README.md
├── package.xml
├── setup.py
├── pyproject.toml
├── apps/
│   ├── run_simulation.launch.py
│   ├── run_baseline_mapping.launch.py
│   ├── run_semantic_mapping.launch.py
│   ├── run_navigation.launch.py
│   ├── run_voice_demo.launch.py
│   ├── run_search_task.launch.py
│   └── run_full_system.launch.py
├── bringup/
│   ├── launch/
│   │   ├── gazebo.launch.py
│   │   ├── robot_base.launch.py
│   │   ├── slam.launch.py
│   │   ├── nav2.launch.py
│   │   ├── perception.launch.py
│   │   ├── interaction.launch.py
│   │   ├── task_manager.launch.py
│   │   └── full_system.launch.py
│   └── rviz/
│       └── robot_system.rviz
├── core/
│   ├── __init__.py
│   ├── state.py
│   ├── event.py
│   ├── action.py
│   ├── task.py
│   ├── constants.py
│   └── topics.py
├── perception/
│   ├── __init__.py
│   ├── lidar/
│   │   ├── scan_adapter_node.py
│   │   └── scan_filter_node.py
│   └── vision/
│       ├── camera_node.py
│       └── yolo_detector_node.py
```

```
| | | object_detector_node.py
| | | person_detector_node.py
| | speech/
| | | asr_node.py
| | | wake_word_node.py
| | fusion/
| | | perception_fusion_node.py
|
| mapping/
| | | __init__.py
| | | slam/
| | | | slam_toolbox_wrapper.py
| | | | map_builder_node.py
| | | semantic/
| | | | semantic_marker_node.py
| | | | person_region_marker_node.py
| | storage/
| | | map_saver_node.py
| | | map_loader.py
| | | map_metadata.py
|
| processing/
| | | __init__.py
| | | strip/
| | | | strip_saved_map_person_free.py
| | | | strip_service_node.py
| | | region/
| | | | region_accumulator.py
| | | | person_region_store.py
| | | filter/
| | | | dynamic_object_filter.py
| | map_refine/
| | | occupancy_map_refiner.py
|
| navigation/
| | | __init__.py
| | | nav2_wrapper_node.py
| | | goal_sender_node.py
| | | navigation_monitor_node.py
| | room_navigation.py
|
| interaction/
| | | __init__.py
| | | nlu/
| | | | intent_parser.py
| | | | command_parser.py
| | | dialog/
| | | | dialog_manager.py
| | | tts/
| | | | tts_node.py
|
| task/
| | | __init__.py
| | | task_manager_node.py
| | | task_planner.py
| | | search_task.py
| | mapping_task.py
```

```
|
|   └─ navigation_task.py
|   └─ patrol_task.py
|
|─ experiment/
|   └─ __init__.py
|   └─ baseline/
|       └─ run_baseline.py
|   └─ semantic_mapping/
|       └─ run_semantic_mapping.py
|   └─ search/
|       └─ evaluate_search.py
|   └─ navigation/
|       └─ evaluate_navigation.py
|   └─ metrics/
|       └─ map_metrics.py
|       └─ experiment_logger.py
|
|─ config/
|   └─ paths.yaml
|   └─ topics.yaml
|   └─ slam.yaml
|   └─ nav2.yaml
|   └─ yolo.yaml
|   └─ asr.yaml
|   └─ nlu.yaml
|   └─ task.yaml
|   └─ experiment.yaml
|
|─ scripts/
|   └─ setup_env.sh
|   └─ build.sh
|   └─ run_simulation.sh
|   └─ run_baseline_mapping.sh
|   └─ run_semantic_mapping.sh
|   └─ run_navigation.sh
|   └─ run_voice_demo.sh
|   └─ run_search_task.sh
|   └─ run_full_system.sh
|   └─ save_map.sh
|   └─ strip_map.sh
|   └─ clean_outputs.sh
|
|─ data/
|   └─ maps/
|       └─ raw/
|       └─ before_strip/
|       └─ after_strip/
|       └─ semantic/
|   └─ logs/
|   └─ results/
|   └─ datasets/
|   └─ recordings/
|
|─ docs/
|   └─ architecture.md
|   └─ runbook.md
|   └─ experiment.md
```

```
|   |— module_design.md
|   |— future_development.md
|
|— legacy/
   |— README.md
```

五、重构原则

请严格遵守以下原则：

1. 不删除不确定用途的文件；
2. 不改变现有核心算法逻辑；
3. 不破坏已有可运行流程；
4. 移动文件后必须同步修改 `import` 路径；
5. 移动 `launch/config/scripts` 后必须同步修改路径引用；
6. 如果某个文件暂时无法归类，放入 `legacy/` 并记录；
7. `shell` 脚本只负责启动、构建、清理，不承载核心业务逻辑；
8. `apps` 和 `bringup` 只负责编排，不写算法；
9. `perception/mapping/processing/navigation/interaction/task` 各层职责要分清；
10. `config` 统一管理参数、`topic`、路径、模型位置；
11. `data` 统一管理输出结果；
12. `docs` 必须说明架构、运行方式和后续扩展方式。

六、重点设计要求

1. baseline 建图流程

需要保留并规范化：

Gazebo/TurtleBot3 启动

- SLAM Toolbox 建图
- 保存地图到 `data/maps/raw` 或 `data/maps/before_strip`
- 记录日志到 `data/logs`
- 记录结果到 `data/results`

2. semantic mapping 实验流程

需要保留并规范化：

Gazebo/TurtleBot3 启动

- SLAM Toolbox 建图
- YOLO/person 检测
- person region 累积
- semantic marker 标记
- 保存 `before_strip`
- `strip` 后处理
- 输出 `after_strip`
- 记录实验参数和结果

3. 未来语音交互流程

请预留模块结构：

用户语音

- ASR
- 文本命令
- NLU intent parser

- task manager
- navigation/search/mapping task
- robot action
- TTS/日志反馈

例如：

“去客厅找杯子”

应该被解析为：

```
intent = search_object
target_object = cup
target_room = living_room
required_modules = navigation + yolo_detection + search_task
```

4. 未来屋内搜索流程

请预留模块结构：

任务输入

- room_navigation
- 到达候选区域
- YOLO object_detector
- 判断是否找到目标
- 若未找到，继续下一个搜索点
- 输出搜索结果

5. YOLO 模块

请将 YOLO 设计为通用视觉检测模块，而不是只服务于 person strip：

perception/vision/yolo_detector_node.py: 通用 YOLO 推理
perception/vision/person_detector_node.py: 人物检测封装
perception/vision/object_detector_node.py: 物体检测封装

这样后续既能识别人，也能识别杯子、椅子、门、包等物体。

6. strip 模块

strip_saved_map_person_free 不能只是散落脚本，应该归入：

processing/strip/

并提供清晰输入输出：

input:

- before_strip map
- person regions
- strip config

output:

- after_strip map
- strip report

7. shell 脚本

请规范 scripts/ 下的一键运行脚本，例如：

./scripts/build.sh

```
./scripts/run_simulation.sh
./scripts/run_baseline_mapping.sh
./scripts/run_semantic_mapping.sh
./scripts/run_navigation.sh
./scripts/run_voice_demo.sh
./scripts/run_search_task.sh
./scripts/run_full_system.sh
./scripts/clean_outputs.sh
```

每个脚本需要：

- 设置必要环境变量；
- source ROS2 和工作区；
- 检查依赖；
- 调用对应 launch；
- 输出清晰日志；
- 不要写死过多绝对路径，尽量读取 config/paths.yaml 或使用项目根目录相对路径。

七、README 需要更新

请更新 README.md，必须包含：

1. 项目简介；
2. 系统目标；
3. 当前已实现功能；
4. 后续扩展功能；
5. 总体架构图；
6. 目录结构说明；
7. 模块职责说明；
8. baseline 建图运行方式；
9. semantic mapping 运行方式；
10. strip 后处理运行方式；
11. navigation 运行方式；
12. voice demo 预留运行方式；
13. search task 预留运行方式；
14. shell 脚本使用方式；
15. 输出目录说明；
16. 参数配置说明；
17. 后续开发规范。

八、docs 文档要求

请创建或更新以下文档：

1. docs/architecture.md

说明整体架构：

```
Application
Bringup
Core
Perception
Mapping
Processing
Navigation
Interaction
Task
Experiment
```

2. docs/runbook.md

说明如何运行：

- 环境准备；
- `build`；
- 启动仿真；
- 启动 `baseline`；
- 启动 `semantic mapping`；
- 启动导航；
- 启动完整系统；
- 清理输出。

3. docs/experiment.md

说明实验设计：

- `baseline`；
- `experiment`；
- `before_strip`；
- `after_strip`；
- 参数冻结；
- 日志记录；
- 输出结果；
- 可复现性。

4. docs/module_design.md

说明各模块接口：

- `perception` 输出什么；
- `mapping` 输入输出什么；
- `processing` 输入输出什么；
- `navigation` 输入输出什么；
- `interaction` 输入输出什么；
- `task manager` 如何调度。

5. docs/future_development.md

说明未来开发路线：

- ASR；
- NLU；
- TTS；
- Nav2；
- room search；
- object detection；
- multi-task planning；
- real robot migration。

九、执行方式

请按以下步骤进行：

第一步：扫描当前仓库结构，输出当前文件分类和存在的问题。

第二步：提出重构计划，说明每类文件移动到哪里。

第三步：执行重构。移动文件、创建目录、更新路径、更新 `import`、更新 `launch`、整理 `shell` 脚本。

第四步：创建或更新 `README` 和 `docs`。

第五步：运行基础检查：

- `Python import` 检查；
- `shell` 脚本语法检查；
- `launch` 文件路径检查；
- `package.xml/setup.py` 是否仍然合理；
- 不要求真实启动 `Gazebo`，但要保证结构和路径尽量正确。

第六步：输出最终总结：

- 改了什么；
- 新架构是什么；
- 现有功能如何运行；
- 未来功能如何接入；
- 哪些文件被放入 `legacy`；
- 哪些地方需要人工确认。

十、特别注意

请不要把项目只整理成“`SLAM` 实验代码”。这个项目最终应该看起来像一个完整室内服务机器人系统，`SLAM`、`YOLO`、`strip` 只是其中一部分。

最终项目应该让别人一眼看出：

1. 系统怎么启动；
2. 建图在哪里；
3. 感知在哪里；
4. `YOLO` 在哪里；
5. 语音识别未来放哪里；
6. 自然语言理解未来放哪里；
7. 导航在哪里；
8. 搜索任务在哪里；
9. `strip` 后处理在哪里；
10. 实验怎么复现；
11. `shell` 脚本怎么运行；
12. 后续开发怎么扩展。

请先分析，再给计划，最后执行重构。